David Nguyen

CSEN 296B-2 — AI-Driven Software Development

Week 3.1 Lab: Refined Requirement Baseline for AstraNotes

This builds on the 14 requirements I wrote in Week 1.2, the 8 user stories and backlog from Week 2.2, and the Working Agreement and DoD from Week 2.1. I didn't try to invent new requirements this week. The goal was to take what I already had and pressure-test it. I went through each requirement and asked: what am I not saying, what would another engineer read differently, and what happens when things go wrong. Ended up at 15 requirements (up by 1 — accessibility was missing), but the real change is that every other requirement got tightened on wording, edge cases, or failure behavior.

Refined Requirement Baseline

Functional Requirements (8)

FR-01 — Create, edit, and delete a note with Markdown support

The system shall allow a user to create a new note with a title and a Markdown body, edit either field, and delete the note. The editor renders a live preview as the user types. All saves persist to Supabase Postgres. Deletion requires a confirmation step. Notes with empty or whitespace-only titles default to "Untitled Note" plus a timestamp rather than being rejected.

> AC: (1) User types `## Hello **world**` and sees a styled heading with bold text in the preview. (2) User deletes a note and is shown a confirmation dialog first. (3) Saved content round-trips to the database and back without losing any formatting. (4) Creating a note with no title assigns "Untitled Note - [timestamp]".
>
> *What changed: Originally I didn't say what happens with empty titles. Saves would have gone through with no title at all and the notes list would've gotten messy. Default-to-Untitled keeps the list readable without forcing the user to type something just to get past validation.*

FR-02 — Folder-based organization with tagging

The system shall allow users to create, rename, and delete folders. Every note belongs to exactly one folder (default: "Uncategorized"). Users can also assign zero or more tags to any note. Folder names must be unique per account. Deleting a folder prompts the user to either move its notes to another folder or delete them along with the folder.

> AC: (1) Moving a note from Folder A to Folder B updates the sidebar counts immediately. (2) Clicking a tag shows only notes with that tag across all folders. (3) Attempting to create a duplicate folder name shows a validation error. (4) Deleting a non-empty folder prompts for a move-or-delete decision.
>
> *What changed: The original didn't prevent two folders named "Work" in the same account, and deleting a folder with notes in it wasn't defined at all. Both are the kind of edge case where users would hit it within the first day and wonder why nothing works.*

FR-03 — Full-text search across title and body

The system shall allow a user to search notes by keyword across both the title and body fields using Postgres tsvector/tsquery indexing. Results are ranked by relevance and highlight matching terms in a snippet preview. An empty search shows all notes. Special characters in the query are sanitized so they do not break the tsquery parser.

> AC: (1) Searching "project kickoff" returns matching notes with highlighted fragments. (2) Results return within 500ms for a library of up to 1,000 notes. (3) Clearing the search bar returns all notes. (4) Searching a query containing `&` or `'` returns results without throwing a parser error.
>
> *What changed: Two ambiguities here. Empty query — does that mean show nothing or show everything? And an apostrophe would've crashed the tsquery parser. Both needed to be called out. The handout literally*

used "search notes quickly" as the example of vague language, so this one was on me.

FR-04 — Email/password authentication

The system shall authenticate users via Supabase Auth with email and password. Sessions are maintained with secure HTTP-only cookies using the Next.js Auth helper. Login errors return a generic message and do not reveal whether the email exists. Passwords must be at least 8 characters. Duplicate registrations with an existing email are rejected without disclosing whether that email is already registered.

AC: (1) New user signs up, confirms via email link, and can log in. (2) Wrong password returns "Invalid login credentials." (3) After 30 minutes of inactivity the session refreshes silently; after 7 days the user must re-authenticate. (4) Attempting to register with an existing email returns the same generic success message as a new registration to avoid user enumeration.

What changed: The original would have leaked which emails exist through different error messages — classic user enumeration. It's a real security hole even at my scale. Fix is just using the same generic response regardless of whether the email is already in the system.

FR-05 — Share a note with read or read-write access

A note owner can share a note with another authenticated user by entering their email. The owner selects read-only or read-write permission. Shared notes appear in the recipient's "Shared with me" section. The owner can revoke access at any time. Sharing with a non-existent email returns a clear "User not found" message. A user cannot share a note with themselves.

AC: (1) Read-only recipient can view but the editor toolbar is disabled. (2) Read-write recipient can edit and save. (3) Revoking access removes the note from the recipient's shared section on next load. (4) Sharing with a non-existent email shows an error. (5) Attempting to share with your own email is blocked.

What changed: The original assumed the recipient always exists and also let you share with yourself, which is technically valid input but isn't what anyone actually wants. Both are blocked now.

FR-06 — Real-time collaborative editing

When two users with read-write access have the same note open, edits from one user appear in the other's editor within 2 seconds via Supabase Realtime channels. If both users edit the same field simultaneously, the system uses last-write-wins resolution at the field level rather than silently dropping either edit. If the Realtime connection drops, the editor stays functional locally and a banner displays "Sync paused - reconnecting." On reconnect, local changes sync to the server using the same last-write-wins strategy.

AC: (1) User A types a sentence; User B sees it within 2 seconds without refreshing. (2) Simultaneous edits to the same field resolve with the most recent write, not a silent drop. (3) A disconnection shows the sync-paused banner and the editor remains writable. (4) Reconnecting syncs changes without duplicating content.

What changed: This was the biggest gap in the whole set. "Real-time editing" sounds obvious until you ask what happens when both users type in the same paragraph at the same time. I was writing around the happy path. Last-write-wins at the field level is a simplification, but at least now the requirement makes a decision instead of hiding one.

FR-07 — Version history with restore

The system shall save a new version snapshot each time a note is explicitly saved. Users can browse the version history, preview any past version in read-only mode, and restore it. Restoring creates a new version entry so the restore action is itself reversible. The system retains the 50 most recent versions per note; older versions are pruned automatically. Version history is visible only to the note owner, not to collaborators with read-write access.

AC: (1) User saves 3 times, opens history, sees 3 entries newest-first with timestamps. (2) Preview shows read-only content at that point in time. (3) Restoring replaces current content and adds a new version entry. (4) After 50 versions, the oldest is pruned. (5) Collaborators with read-write access do not see the history

panel.

What changed: As written, version history would've grown unbounded and I never said whether collaborators could see or roll back my history. 50-version cap and owner-only visibility cleans up both.

FR-08 — Export notes as Markdown

The system shall allow a user to export any single note as a .md file download. The exported file contains the raw Markdown source. A bulk export option downloads all notes in a selected folder as a .zip archive. Filenames are derived from note titles and sanitized to remove characters that are invalid on common operating systems (`/`, `\`, `:`, etc.). Exporting an empty folder shows a message instead of producing an empty zip.

AC: (1) Exporting a note with headings, code blocks, and links produces a valid .md file that renders correctly in any Markdown viewer. (2) Bulk export of a folder with 5 notes produces a .zip containing 5 .md files. (3) A note titled "Q3 / Budget & Forecast" exports as a filename with the slashes and ampersand sanitized. (4) Exporting an empty folder shows "This folder has no notes to export."

What changed: A note titled "Q3 / Budget & Forecast" would've produced a broken filename on Windows. Empty folders would've exported an empty zip with no message. Small stuff, but exactly the kind of thing that makes the app feel buggy.

Non-Functional Requirements (4)

NFR-01 — Page load performance

The authenticated dashboard shall reach First Contentful Paint in under 1.5 seconds on a standard broadband connection for a note library of up to 1,000 notes. The note list is rendered via Next.js server components so the initial HTML includes real content rather than a loading spinner.

AC: (1) Lighthouse desktop performance score is at least 90. (2) Cold load with 50 notes shows the note list in under 1.5s measured by FCP. (3) With 1,000 notes the initial render still completes under 2s.

What changed: I had "under 1.5 seconds" but never said at what note count. "Fast" at 10 notes and "fast" at 10,000 notes aren't the same thing — a perf target without a load assumption isn't testable. Added the 1,000-note upper bound to line it up with the search target in FR-03.

NFR-02 — Mobile responsiveness

All primary views (note list, editor, search, folder sidebar) shall be usable on viewports down to 375px wide. The sidebar collapses into a hamburger menu on mobile. The editor toolbar wraps and all touch targets are at least 44x44 pixels. The side-by-side editor/preview layout collapses to a tabbed toggle on mobile since both panes will not fit in 375px.

AC: (1) At 375px the sidebar is behind a menu icon. (2) No horizontal scrollbar on any primary view. (3) All toolbar buttons meet the 44px minimum. (4) On mobile, editor and preview are accessed via a toggle, not shown side by side.

What changed: I had said "responsive" but never addressed what happens to the side-by-side editor/preview at phone width. Two panes don't fit in 375px no matter how you shrink them. Tabbed toggle is the least-bad option.

NFR-03 — Testability by design

The storage layer, access control logic, and sync service shall be separated into distinct modules so each can be unit-tested independently. Integration tests can run against a local Supabase instance without depending on production data. Each module has at least one isolated unit test.

AC: (1) Running the test suite against a seeded local Supabase instance produces passing results for note CRUD, RLS enforcement, and version history without touching production. (2) Storage, access control, and sync modules each have at least one unit test that runs without the others loaded.

What changed: Already pretty solid. Just tightened the per-module test criterion so it's auditable instead of vibes.

NFR-04 — Accessibility baseline

The interface shall meet WCAG 2.1 Level AA on the core flows (note creation, editing, search, navigation). Text contrast ratios meet AA thresholds. All interactive elements are reachable by keyboard. The editor, sidebar, and search results are navigable with a screen reader using semantic HTML and ARIA labels where needed.

AC: (1) An automated axe-core scan on the dashboard, editor, and search views produces zero critical violations. (2) All primary actions (create, save, delete, search, share) are reachable using only keyboard input. (3) Text elements meet a 4.5:1 contrast ratio against their background.

What changed: This one's new. Accessibility was implicit in the original set, which is exactly the "assumed but never written" pattern the handout warned about. If it's not in the baseline I won't test for it, so it goes in.

Security, Privacy, and Reliability Requirements (3)

SEC-01 — Row-level security as the trust boundary

Supabase RLS policies shall be enabled on the notes, folders, tags, note_shares, and note_versions tables. A user can only read or modify rows where they are the owner or an explicitly granted collaborator. No API route or server action bypasses RLS. The owner_id column is the single trust boundary for all data access. Cross-user access is prevented at the database layer, not the application layer.

AC: (1) An integration test that authenticates as User A and queries User B's notes returns zero rows. (2) No API route uses the Supabase service role key outside of explicit admin paths. (3) Every table with user data has at least one RLS policy enforcing owner-or-shared access.

What changed: The original said access control "should be enforced" but didn't say where. That's the kind of drift that turns into a vulnerability when someone (me, six weeks from now) adds a route that bypasses RLS because it was convenient. Explicit trust boundary at the DB layer closes that off.

SEC-02 — Encrypted data in transit

All client-server communication shall occur over HTTPS/TLS. Supabase connections use SSL by default, and the Next.js deployment enforces HTTPS with automatic certificate management. No API endpoint is accessible over plain HTTP. HSTS is enabled on the deployment.

AC: (1) Every request to the production domain over HTTP is redirected to HTTPS. (2) The response headers include a `Strict-Transport-Security` header. (3) No API route logs show plain-HTTP traffic.

What changed: I had HTTPS but no HSTS, which still leaves a first-visit downgrade attack open. Small fix, real coverage gap.

SEC-03 — Graceful degradation on service outage

If the Supabase Realtime connection drops during a collaborative editing session, the editor shall remain functional in local-only mode and the user's unsaved changes shall not be lost. A visible banner notifies the user that sync is paused. On reconnect, local changes are reconciled using the same last-write-wins strategy defined in FR-06.

AC: (1) Simulating a connection drop leaves the editor writable. (2) Local edits during the outage are synced on reconnect. (3) The sync-paused banner is visible throughout the outage. (4) Reconnection does not produce duplicate content.

What changed: The original said edits would be "reconciled" on reconnect but never said how, which is the same conflict problem as FR-06. Pointing this requirement at the FR-06 strategy means I don't end up with two conflicting decisions six weeks from now.

Ambiguity Review

Going through the original set, these are the places where two engineers would have read the same requirement differently:

- FR-06 never defined what happens when two users type in the same field at the same time. "Real-time editing" and "no overwriting" both sound reasonable, but they contradict each other the second both people touch the same paragraph. Fix: last-write-wins at the field level, stated explicitly.
- FR-03 didn't say what happens when the search bar is empty or when the query has special characters. Does clearing it mean "show nothing" or "show everything"? Should an apostrophe crash the tsquery parser? Neither was answered. Fix: empty returns all notes, special characters get sanitized.
- SEC-03 said edits would be "reconciled" on reconnect but never said how. That's the same conflict problem as FR-06 and should use the same answer. Fix: explicitly reuse the FR-06 last-write-wins rule.
- NFR-01 said "under 1.5 seconds" but never said at what note count. A performance target without a load assumption isn't meaningful. Fix: 1,000-note upper bound, matching the search target in FR-03.

Edge-Case Review

Edge cases the original set either missed or handled with implicit assumptions that should've been explicit:

- Empty or whitespace-only titles (FR-01): now defaults to "Untitled Note" + timestamp.
- Duplicate folder names (FR-02): two folders called "Work" were allowed. Now enforced as unique per account with a validation error.
- Sharing with a non-existent email (FR-05): now returns "User not found" instead of failing silently or creating a dangling share record.
- Self-sharing (FR-05): entering your own email in the share dialog is technically valid but nobody wants that. Now blocked.
- Special characters in note titles on export (FR-08): a title like "Q3 / Budget & Forecast" would've produced a broken filename on Windows. Now sanitized.
- Exporting an empty folder (FR-08): would've produced an empty zip with no message. Now shows "This folder has no notes to export."
- Preview pane on mobile (NFR-02): side-by-side editor and preview don't fit at 375px. Now hidden behind a toggle on mobile.
- Version history visibility for collaborators (FR-07): wasn't defined. Now limited to the owner only.
- Duplicate email registration (FR-04): would've leaked which emails exist. Now returns the same generic response as a new registration.

Functional vs Non-Functional: What I Moved

Two things were sitting in the wrong category and I moved them:

- The "fast search" target was tucked inside FR-03 as part of the functional requirement. Speed is a quality constraint, not a behavior. The 500ms target stays with FR-03 as an acceptance criterion, but the broader performance expectation belongs in NFR-01.

- "Notes are private by default" was written originally as a behavior (functional), but it's really a policy statement about the trust boundary. It belongs with SEC-01 (RLS), which is where the actual enforcement happens.

How AI Helped Refine the Requirements

I used Claude as a reviewer this week instead of a drafter. The handout called for pressure-testing and that matched how I've been using AI in my day job anyway. I pasted each requirement one at a time and asked the same three things: what does this not say, what would two engineers disagree on, and what happens in the failure case.

The most useful pushback came on FR-06. Claude asked the exact question I'd been avoiding: "what happens when both users type in the same paragraph at the same time?" I didn't have an answer. That's when I realized the original was written around the happy path with no conflict strategy at all. Last-write-wins is a simplification, but the point is the requirement now makes a decision instead of hiding one.

Same kind of questioning caught the security gaps. Asking "how does your login error behave for valid vs invalid emails" exposed the user enumeration in FR-04. Asking "what does HTTPS enforcement actually cover" surfaced the missing HSTS in SEC-02. These are the kind of problems I would've shipped without noticing.

Not everything it suggested stuck. It pushed hard on end-to-end encryption, full offline-first architecture, and enterprise SSO. All reasonable features but none of them realistic for a solo 10-week project. I left them out. The goal this week was to tighten the existing baseline, not to expand it.

Final count: 15 requirements (8 functional, 4 non-functional, 3 security/privacy). Net change from the original 14: one new requirement (NFR-04, accessibility), plus tighter wording and added edge cases on every other one. Still fits the scope of a solo quarter project.